System-level scheduling of rotating-chamber semiconductor cluster tools: A source-configurable two-stage MILP and structure-aware exact search

Lizhen Shao[a,*], Lincong Wei[a], Wei Wang[a]

[a]*Department of Control Science and Engineering, University of Science and Technology Beijing, No. 30 Xueyuan Road, Haidian District, Beijing 100083, China*

Abstract

Rotary-chamber cluster tools must coordinate product wafers, reusable dummy wafers, cleaning, robots, and chamber timing under full occupancy. We develop a source-configurable two-stage MILP that uses one internal model for a single-source specialization and dual-source mixed flow and covers both 4×1 parallel and 2×2 chained processing. Stage one minimizes end-to-end makespan; stage two preserves the discrete schedule and current makespan while improving waits and chamber cadence. Finite-time exact search is enhanced by a structure-preserving binary-skeleton (SPBS) initial solution and the role-aware hierarchical cut selector RDMCT-HBS. Exact equal-parameter comparisons show that pure 4×1 has the shortest makespan for recipe-unconstrained 8- and 16-wafer lots. Under the same 600-s budget, the two-stage workflow reduces four waiting and cadence measures by 39.70%–94.05%. SPBS raises incumbent availability from 51.67% to 100% and lowers mean PDI by 26.19%. Among seven nominal configurations sharing SPBS, RDMCT-HBS records the lowest observed mean PDI and solution time. The framework provides reproducible system-level scheduling while retaining solver-checked feasibility, bounds, and optimality status.

Keywords: semiconductor manufacturing, cluster tool, scheduling, mixed-integer linear programming, branch-and-cut, warm start

1. Introduction

Semiconductor manufacturing depends on capital-intensive, highly automated process equipment. Equipment utilization and production cadence depend not only on chamber processing times but also on coordination among wafer transport, load-lock switching, chamber occupancy, and control logic. A cluster tool (CT) integrates process modules, load locks, and wafer-handling robots in one controlled environment; residency limits, chamber cleaning, and activity-time variation further shrink the feasible scheduling space (Pan et al., 2018). CT scheduling is therefore an equipment-level manufacturing-system coordination problem rather than a generic parallel-machine problem.

Photolithography, etching, and deposition are three central stages of semiconductor manufacturing. In thin-film deposition, plasma-enhanced chemical vapor deposition (PECVD) for advanced processes is a major focus of equipment development. As throughput and floor-space requirements rise, conventional CTs no longer meet the full complexity of PECVD operation. Manufacturers therefore place several processing spaces inside a module and use chamber rotation to exchange the accessible positions, allowing several wafers to be processed while retaining robotic access to every station (Gu et al., 2024).

Carbon-containing films are difficult to clean, and the multi-space rotary chamber considered here must operate without empty stations. Reusable dummy wafers protect otherwise vacant positions. Product wafers enter from the atmospheric side, whereas a reusable-dummy storage module is mounted directly in the vacuum domain; the two flows merge at the chamber and compete for the same robot. Full occupancy and dummy filling create two representative modes: 4×1 processes four positions in parallel for one operation, whereas 2×2 processes two wafer pairs through a two-operation chain. Figure 1 shows the topology. If the recipe does not prescribe a mode, pure 4×1, pure 2×2, and mixed operation must be compared at the same wafer count and time parameters to separate intrinsic mode efficiency from mixed-flow coordination cost.

CT scheduling first developed around throughput analysis and residence-time feasibility. Perkinson et al. (1994) related process and transport times to the maximum throughput of a single-wafer CT using an analytical model. Rostami and Hamidzadeh (2002) formulated dual-arm scheduling with both process-module and transport-module residency constraints, combining linear programming with problem-specific heuristics. Wu and Zhou (2010) used a timed Petri-net steady schedule to derive closed-form schedulability conditions and an optimal periodic schedule. Qiao et al. (2012) then extended Petri-net analysis to bounded activity-time variation and obtained an upper bound on wafer sojourn delay for single-arm tools.

Subsequent work broadened the process routes and production mix. Qiao et al. (2014) modeled atomic-layer-deposition revisiting and residency constraints in a dual-arm CT and derived an optimal one-wafer cyclic schedule. Wang et al. (2020) addressed several wafer types sharing a dual-arm tool under residency limits. Wang et al. (2025) introduced a virtual-cell formulation that selects both the number of active process modules

*Corresponding author.

Email addresses: lshao@ustb.edu.cn (Lizhen Shao), m202420836@xs.ustb.edu.cn (Lincong Wei)

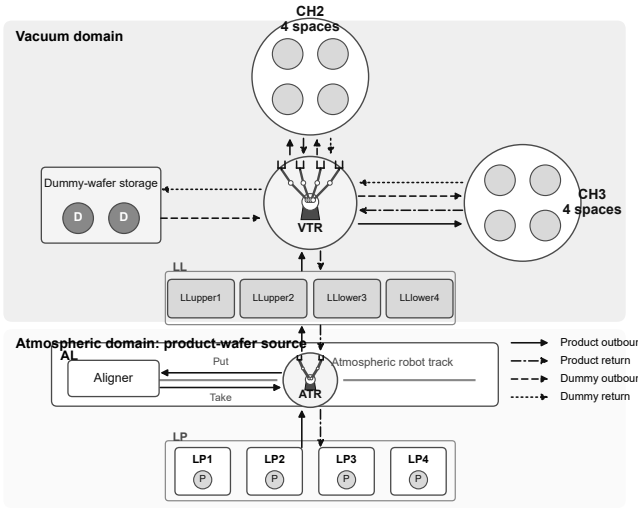


Figure 1: Dual-source mixed-flow cluster tool. The LP bank is centred below the vertically aligned ATR and VTR. The dual-hand ATR performs LP pickup, aligner place–then–pick handling, and product return; the VTR has four hands, and the fork grippers of both robots face the chamber-access direction. Solid, dash-dot, dashed, and dotted arrows denote product outbound, product return, dummy outbound, and dummy return routes. Storage–VTR, VTR–rotary-chamber, and ATR–LP links all show separate outbound and return paths. The tool may contain multiple rotary chambers; two four-space chambers are illustrated.

and robot waiting times instead of assuming that all modules must be used. At the manufacturing-system level, Lee et al. (2018) linked waiting-time violations to yield, buffer design, and upstream-machine control in a Bernoulli production line, showing why equipment scheduling must retain time-window information when system-level performance is evaluated.

Cleaning added a second line of constraints because a maintenance action occupies chambers and changes the robot sequence. Kim et al. (2013) generated feasible dual-arm robot-task sequences for periodic chamber cleaning and used heuristics when complete enumeration became expensive. Yu and Lee (2020) combined partial loading with robot-task timing control to reduce both cycle time and wafer delay in dual-arm tools with cleaning. Qiao et al. (2022) represented cleaning occupancy with virtual wafers and optimized a residency-constrained dual-arm CT using binary integer programming. Li et al. (2022) linked cleaning triggers to chamber condition and derived feasible schedules for time-constrained single-arm tools.

Recent studies have moved toward richer equipment configurations and mixed production. Gu et al. (2024) used linear programming to obtain minimum steady-state cycles for a four-finger robot and four-space modules. Yang et al. (2025) used MIP to determine release and robot-task sequences jointly for two wafer types. These advances still leave a distinct finite-batch problem in which two external sources feed the same vacuum domain, reusable dummies enforce full occupancy, and 4×1 and 2×2 rotary recipes coexist.

Learning-based CT controllers operate at a different decision layer. Kim and Lee (2025) combined deep reinforcement learning with look-ahead search to select non-cyclic actions. Hong et al. (2026) trained DDPG in a discrete-event model to adjust PVD process times. Here, processing times are inputs and the output must be a complete schedule with MILP feasibility checks, bounds, and an optimality status. Learning-enhanced exact optimization therefore provides a more suitable interface. Tang et al. (2020) formulated cut selection as reinforcement learning. Huang et al. (2022) learned candidate-cut rankings. Paulus et al. (2022) used a strong look-ahead rule to generate imitation targets. Wang et al. (2023b) used a hierarchical policy to select both the cut budget and an ordered cut subset.

Two gaps follow. At the equipment level, no finite-horizon model unifies external source topology, full-occupancy recipes, reusable dummies, cleaning, and shared transport. At the solver level, generic cut features do not distinguish discrete equipment choices, continuous event times, and their coupling. This paper carries equipment structure through formulation, initialization, and cut selection. Its contributions are:

1. a source-configurable two-stage MILP that expresses a single-source specialization and dual-source mixed flow with the same internal model, covering $4{\times}1/2{\times}2$ processing, dummy reuse, cleaning, residency, and shared resources;

2. a throughput-first, operational-quality-second mechanism that reduces waits and cadence variation while preserving the discrete schedule and current makespan; and

3. an SPBS initial solution and the RDMCT-HBS cut selector, which improve finite-time primal and dual search without changing the MILP feasible region or proof logic.

2. Industrial problem and two-stage MILP

2.1. Model boundary, material routes, and notation

We formulate a finite-batch offline schedule. Inputs are the number and mode of product wafers, equipment action times, cleaning interval, residency limits, load-lock capacity, and dummy inventory. Outputs are complete routes for every product and dummy wafer, start and end times for each action, and the order of actions competing for a physical resource. Times are known within one solve and actions are non-preemptive. No buffer is introduced beyond the load ports, aligner, load locks, chambers, and dummy storage.

A wafer pair (PW pair) is a physical two-wafer transport and chamber-occupancy unit. Its members may be product+product (product PW pair), dummy+dummy (dummy PW pair), or product+dummy (mixed PW pair). Product wafers are preferentially paired by mode and index; an odd tail product may be paired with one dummy, while full-occupancy filling and cleaning may use dummy PW pairs. The sets $\mathcal{P}^F$ and $\mathcal{P}^L$ index product-containing PW pairs, including product PW pairs and any required mixed PW pairs. Alignment and residency remain wafer-level operations; the two members share a time window only on a selected two-hand transfer or the same chamber side. Preprocessing maps equivalent product-containing PW pairs to canonical compatible positions. This removes label-swapping symmetry but leaves active batches or chain positions, physical dummy identities, paired transfers, resource order, and all

event times to the MILP. Legal cleaning segments are generated from the cleaning interval and complete recipe boundaries, after which the MILP activates the cleaning tasks and schedules their times.

A product follows LP–ATR–AL–ATR–upper load lock–VTR–rotary chamber–VTR–lower load lock–ATR–LP: the ATR first places the wafer on the aligner, then retrieves it for transfer into the vacuum domain. A dummy travels from vacuum storage through the VTR to the chamber and returns through the VTR after use. A regular all-product 4×1 batch contains two product PW pairs, hence four product wafers, and processes all four positions synchronously once before last-in–first-out unloading; only the last incomplete batch may use a dummy or mixed PW pair to fill all positions. In 2×2, every product-containing PW pair occupies two adjacent process cycles so that each of its product members is processed twice. For three product PW pairs, a dummy PW pair D is loaded first, the chamber rotates, and P1 is loaded before D+P1 is processed. The chain then slides through P1+P2, P2+P3, and P3+D, with a process after each slide; P1, P2, and P3 are each processed twice before all wafers are unloaded. Figures 2 and 3 show the event sequences.

Tables 1 and 2 list every symbol in the core formulation and distinguish sets/indices, input constants, fixed binary aliases, binary decisions, continuous decisions, and derived quantities. In the implementation, x^F and x^L are continuous $[0, 1]$ aliases fixed to zero or one by equality; this preserves uniform export names without creating extra branching decisions. To fit the official double-column template, the complete notation is divided into two full-width tables without deleting entries.

2.2. Source interfaces, operation activation, and time relations

The fixed incidence parameter satisfies

$$\sum_{s \in \mathcal{S}} \chi_{is} = 1, \qquad i \in \mathcal{I}. \tag{1}$$

Equation (1) assigns each material unit to exactly one source. Setting $|\mathcal{S}| = 1$ gives the single-source tool; using two interfaces and assigning products and dummies to different interfaces gives the dual-source tool.

If $\mathcal{A}^{\text{int}}$ is the internal network and $\mathcal{A}_s$ contains release and return arcs for source s, then

$$\mathcal{A} = \mathcal{A}^{\text{int}} \cup \left(\bigcup_{s \in \mathcal{S}} \mathcal{A}_s \right). \tag{2}$$

Equation (2) means that one- and two-source tools differ only at route endpoints; processing, cleaning, shared resources, and timing use the same internal model, so a dual-source tool is not the sum of two independent single-source schedules.

Canonical pair positions satisfy

$$\begin{aligned} x^F_{pcbl} &= \bar{x}^F_{pcbl}, \quad \sum_{c,b,l} x^F_{pcbl} = 1 \quad (p \in \mathcal{P}^F), \\ x^L_{pck} &= \bar{x}^L_{pck}, \quad \sum_{c,k} x^L_{pck} = 1 \quad (p \in \mathcal{P}^L). \end{aligned} \tag{3}$$

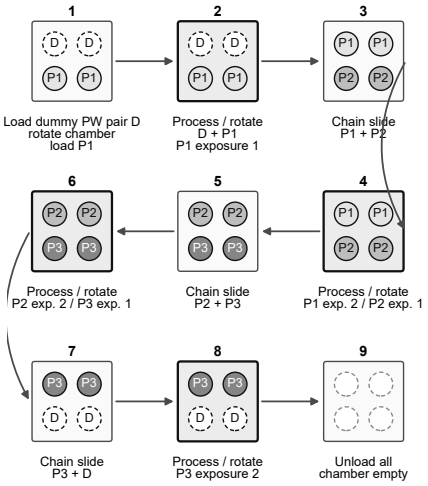


Figure 2: 2×2 chained processing. A PW pair may be P+P, D+D, or P+D; here each P*i* is a P+P product PW pair and D is a D+D dummy PW pair. Step 1 loads D, rotates the chamber, and loads P1; Step 2 processes and rotates D+P1; Step 3 slides the chain to P1+P2. Subsequent slide–process events make P1, P2, and P3 participate in two adjacent process cycles, so every product wafer is processed twice before all wafers are unloaded.

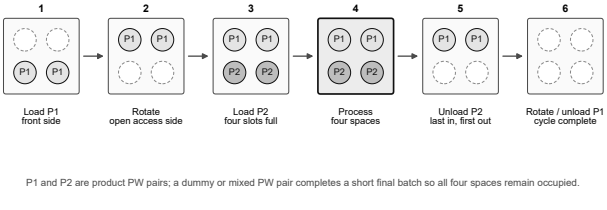


Figure 3: 4×1 parallel processing. A regular all-product batch uses two product PW pairs, comprising four product wafers, and receives one synchronous full-occupancy process before last-in–first-out unloading; a dummy or mixed PW pair can complete the final short batch.

Equation (3) assigns each wafer pair to one compatible position and fixes equivalent locations to canonical indices. It removes label symmetry, not event times or resource order.

A position containing a pair must be active, and candidate positions are used consecutively:

$$\begin{aligned} x^F_{pcbl} &\le u^F_{cb}, \quad u^F_{c,b+1} \le u^F_{cb}, \\ x^L_{pck} &\le u^L_{ck}, \quad u^L_{c,k+1} \le u^L_{ck}. \end{aligned} \tag{4}$$

Equation (4) activates any batch or chain position containing a product and excludes a nonphysical inactive position followed by an active one.

For each candidate operation o, activation and duration satisfy

$$0 \le s_o \le M b_o, \qquad 0 \le e_o \le M b_o, \qquad e_o = s_o + d_o b_o. \tag{5}$$

Equation (5) sets inactive operation times to zero and gives an active operation duration d_o. It applies to robot moves, alignment, pump/vent, rotation, product processing, dummy transport, and cleaning.

Table 1: Core notation I: sets, indices, and input parameters

Symbol	Domain and role	Meaning
$\mathcal{S}, \mathcal{I}$	sets/indices	Material-source interfaces and material units; $i \in \mathcal{I}$ may be a product or dummy demand.
$\mathcal{W}$	set/index	Product wafers, indexed by w.
$\mathcal{P}^F, \mathcal{P}^L$	sets/indices	Product-containing PW pairs assigned to 4×1 and 2×2; they include P+P product pairs and any required P+D mixed pairs, indexed by p.
$\mathcal{C}$	set/index	Rotary process chambers, indexed by c.
$\mathcal{B}_c, \mathcal{K}_c$	ordered sets/indices	Candidate 4×1 batches b and 2×2 product-chain positions k in chamber c.
$\mathcal{L} = \{1, 2\}$	set/index	Front and back sides of a 4×1 batch.
$\mathcal{Q}_c, \mathcal{E}_c$	ordered sets/indices	2×2 process cycles q and cleaning segments e in chamber c.
$\mathcal{V}_c$	set/index	Union of all 4×1 batches and 2×2 cycles in chamber c.
$\mathcal{O}, \mathcal{A}$	sets/indices	Candidate physical operations and required process-precedence arcs.
$\mathcal{R}, \Lambda_r$	sets/indices	Shared resources and physical slots of resource r.
$\mathcal{J}_c^D, \mathcal{T}_c$	sets/indices	Dummy-wafer demands and reusable physical dummies of chamber c.
χ_{is}	known binary constant	Whether material unit i belongs to source interface s.
$\bar{x}^F_{pcbl}, \bar{x}^L_{pck}$	known binary constants	Canonical pair-to-4 × 1-side or pair-to-2 × 2-position assignment.
$q_c^-(k), q_c^+(k)$	known nonnegative integer maps	First and second process cycles of 2×2 chain position k, with $q_c^+(k) = q_c^-(k) + 1$.
$\bar{z}_{ce}, \bar{a}^F_{cbe}, \bar{a}^L_{cqe}$	known binary constants	Active cleaning segments and pre-generated membership of batches or cycles.
$H, \bar{Q}_c$	nonnegative integer constants	Maximum cycles between cleanings and generated 2×2 cycles in chamber c.
$d_o, \delta_{oo'}$	nonnegative constants	Duration of operation o and minimum lag on arc (o, o').
d_v^F, d^L, r_c	nonnegative constants	4×1 stage duration, 2×2 process time, and chamber half-rotation time.
$\rho^r_{oo'}$	nonnegative constant	Empty travel or reset time from operation o to o' on resource r.
$M, \bar{R}_{wr}$	positive constants	Valid horizon-based big-M and maximum residency of wafer w in restricted region r.
κ_o, Γ_o	integer/set constants	Slots required by operation o and the slots it may use.
$\lambda_1, \ldots, \lambda_4$	nonnegative constants	Stage-two weights for total wait, maximum wait, cadence deviation, and soft-limit excess.
$\bar{\omega}_\alpha, \varepsilon_C$	nonnegative constants	Soft limit for wait α and the stage-two makespan tolerance.
$\mathcal{F}, \mathcal{D}_{\text{fix}}$	sets	Full MILP feasible region and discrete indices fixed in stage two.
$\mathcal{Q}^w, \mathcal{I}_h$	sets/indices	Wait quantities and process starts admitting cadence second differences.

For every required process arc $(o, o') \in \mathcal{A}$,

$$b_{o'} \le b_o, \qquad s_{o'} \ge e_o + \delta_{oo'} - M(2 - b_o - b_{o'}). \tag{6}$$

Equation (6) activates a successor only with its predecessor and, when both exist, starts it after the predecessor and minimum lag. These arcs connect product routes, dummy routes, and load–process–unload chains.

2.3. Occupancy and event order in the two processing modes

For 4×1, let $A^F_{cb} = \sum_{p,l} x^F_{pcbl}$ be the number of product pairs assigned to batch (c, b). Full occupancy and dummy filling are

$$\begin{aligned} A^F_{cb} &= \sum_{p \in \mathcal{P}^F} \sum_{l \in \mathcal{L}} x^F_{pcbl}, \\ \sum_{p \in \mathcal{P}^F} x^F_{pcbl} + f^F_{cbl} &= u^F_{cb} \qquad (l \in \mathcal{L}), \\ \sum_{l \in \mathcal{L}} f^F_{cbl} &= 2u^F_{cb} - A^F_{cb}. \end{aligned} \tag{7}$$

Equation (7) gives each active 4×1 batch two sides, each occupied by one product pair or one dummy pair; all four chamber positions are therefore filled at process start.

Active batches are consecutive and dummy filling is restricted to the last active batch:

$$\begin{aligned} A^F_{cb} &\ge u^F_{cb}, & u^F_{c,b+1} &\le u^F_{cb}, \\ A^F_{cb} &\ge 2u^F_{c,b+1}, & t^F_{cb} &\le u^F_{cb}, \\ t^F_{cb} &\le 1 - u^F_{c,b+1}, & t^F_{cb} &\ge u^F_{cb} - u^F_{c,b+1}. \end{aligned} \tag{8}$$

Equation (8) requires at least one product pair in an active batch and two product pairs if a later batch is active. The last three inequalities set $t^F_{cb} = 1$ exactly at the active tail; the final candidate uses $t^F_{cB} = u^F_{cB}$.

Let FL, BL, P, BU, FU denote front load, back load, processing, back unload, and front unload. Then

$$\begin{aligned} e^{F,\nu}_{cb} &= s^{F,\nu}_{cb} + d^F_\nu u^F_{cb} && (\nu \in \{FL, BL, P, BU, FU\}), \\ s^{F,BL}_{cb} &\ge e^{F,FL}_{cb} + r_c - M(1 - u^F_{cb}), \\ s^{F,P}_{cb} &= e^{F,BL}_{cb}, \\ s^{F,BU}_{cb} &\ge e^{F,P}_{cb} - M(1 - u^F_{cb}), \\ s^{F,FU}_{cb} &\ge e^{F,BU}_{cb} + r_c - M(1 - u^F_{cb}). \end{aligned} \tag{9}$$

Equation (9) loads the front side, rotates to load the back, starts synchronous processing after the back load, and then unloads the back side before rotating and unloading the front. Consecutive batches are linked by $s^{F,FL}_{c,b+1} \ge e^{F,FU}_{cb} - M(1 - u^F_{c,b+1})$.

For 2×2, each product pair occupies one ordered chain position:

$$u^L_{ck} = \sum_{p \in \mathcal{P}^L} x^L_{pck}, \qquad u^L_{c,k+1} \le u^L_{ck}, \qquad a^L_c = u^L_{c1}. \tag{10}$$

Equation (10) creates a one-to-one relation between product pairs and consecutive chain positions and uses a^L_c to indicate whether chamber c contains a 2×2 chain.

Let $q_c^-(k)$ and $q_c^+(k)$ denote the first and second process cycles of chain position k. The pair–cycle incidence is

$$\begin{aligned} q_c^+(k) &= q_c^-(k) + 1, \\ \mu^L_{pcq} &= \sum_{\substack{k \in \mathcal{K}_c: \\ q \in \{q_c^-(k), q_c^+(k)\}}} x^L_{pck}, \\ \sum_{q \in \mathcal{Q}_c} \mu^L_{pcq} &= 2 \sum_{k \in \mathcal{K}_c} x^L_{pck}. \end{aligned} \tag{11}$$

Equation (11) maps every assigned 2×2 product-containing PW pair to exactly two adjacent process cycles. Every product

Table 2: Core notation II: decision variables, fixed aliases, and derived quantities

Symbol	Domain and role	Meaning
x^F_{pcbl}, x^L_{pck}	fixed continuous $[0,1]$ aliases	Product-containing PW-pair placement on a 4×1 batch side or at a 2×2 chain position.
μ^L_{pcq}	fixed continuous $[0,1]$ alias	Whether 2×2 product-containing PW pair p is processed in cycle q of chamber c.
$u^F_{cb}, f^F_{cbl}, t^F_{cb}$	binary variables	4×1 batch activation, dummy filling on a side, and final active batch.
$u^L_{ck}, a^L_c, v^L_{cq}, h^L_{ck}$	binary variables	2×2 position, chain, process-cycle, and chain-tail activation.
$z_{ce}, a^F_{cbe}, a^L_{cqe}, q^{\mathrm{cl}}_{ch}$	fixed-binary aliases/variables	Cleaning segments, task membership, and cleaning-task activation.
$q_j, g_{j\tau}$	binary variables	Dummy demand and assignment to physical dummy τ.
$b_o, y^r_{oo'}$	binary variables	Operation activation and whether o precedes o' on resource r.
$h_{o\ell}$	binary slot variable	Whether operation o occupies physical slot ℓ.
γ_{pa}	binary variable	Whether pair p uses one paired transfer on robot route a.
s_o, e_o	nonnegative variables	Start and end times of operation o.
$s^{F,v}, e^{F,v}, s^L, e^L$	continuous aliases	Stage-specific notation for s_o, e_o in the two modes.
A^F_{cb}, N_{ce}	derived continuous/integer quantities	Product-pair count in a 4×1 batch and active cycles in a cleaning segment.
a_{vce}	fixed binary alias	Whether task $v\in\mathcal{V}_c$ belongs to cleaning segment e.
$C_w, C_{\max}$	nonnegative variables	Product completion time and lot makespan.
$\omega_\alpha, L_{\mathrm{sum}}, L_{\max}$	nonnegative variables/derived quantities	Stage-two wait or idle quantities, their sum, and their maximum.
$\Delta_{ceq}, v_{ceq}, \eta_\alpha$	derived/nonnegative variables	Cadence second difference, its absolute-value bound, and soft wait excess.
h_{ceq}, Φ	continuous alias/derived objective	Start of the qth process in a segment and stage-two schedule-quality objective.
$\xi, \widehat{\xi}, \widehat{C}$	vector/stage constants	Current variables, stage-one incumbent, and incumbent makespan.

member inherits the same μ^L_{pcq} incidence, so a product is not omitted even when an odd tail uses a P+D mixed PW pair. The equation also distinguishes the modes explicitly: a 2×2 product-containing pair spans two cycles, whereas two 4×1 side pairs are processed once in one full batch.

The generated layout supplies the number $\bar{Q}_c$ of process cycles, and h^L_{ck} identifies the chain tail:

$$\begin{aligned} v^L_{cq} &= \begin{cases} a^L_c, & q\le \bar{Q}_c,\\ 0, & q>\bar{Q}_c, \end{cases}\\ \sum_k h^L_{ck} &= a^L_c, \qquad h^L_{ck}\le u^L_{ck}, \qquad h^L_{ck}+u^L_{c,k+1}\le 1,\\ h^L_{ck} &\ge u^L_{ck}-u^L_{c,k+1}. \end{aligned} \tag{12}$$

Equation (12) gives $n+1$ process cycles for a segment with n product-containing PW pairs: the head is dummy PW pair+P1, internal cycles are Pi+P$i+1$, and the tail is Pn+dummy PW pair. The final relations make h^L_{ck} one only at the last active product position.

For a legal chain segment $(q_0,\dots,q_T)$, let H_0, B_q, and T_0 denote head loading, internal exchange, and tail unloading. Then

$$\begin{aligned} e^L_{cq}-s^L_{cq} &= d^L v^L_{cq},\\ s^L_{cq_0} &= e^{H_0}_c,\\ s^B_{cq} &\ge e^L_{c,q-1}-M(1-a^L_c), \qquad s^L_{cq}=e^B_{cq} \quad (q_0<q\le q_T),\\ s^{T_0}_c &\ge e^L_{cq_T}-M(1-a^L_c). \end{aligned} \tag{13}$$

Equation (13) begins the first process immediately after head loading, performs an internal exchange only after the previous cycle, and unloads the tail only after the last process. It directly realizes dummy+P1, P1+P2, …, Pn+dummy.

2.4. Cleaning and reusable dummy wafers

Cleaning segments are generated from H and complete process boundaries and enter the MILP as fixed binary aliases:

$$z_{ce}=\bar{z}_{ce}, \qquad a^F_{cbe}=\bar{a}^F_{cbe}u^F_{cb}, \qquad a^L_{cqe}=\bar{a}^L_{cqe}v^L_{cq}. \tag{14}$$

Equation (14) prevents arbitrary cleaning locations: 4×1 may split only at batch boundaries and 2×2 only after a complete chain segment, while the MILP still schedules process and cleaning times.

Let N_{ce} be the number of active process cycles in segment e:

$$\begin{aligned} N_{ce} &= \sum_b a^F_{cbe}+\sum_q a^L_{cqe}, \qquad z_{ce}\le N_{ce}\le Hz_{ce},\\ q^{\mathrm{cl}}_{ch} &= z_{c,h+1}. \end{aligned} \tag{15}$$

Equation (15) places at least one and at most H process cycles in an active segment and activates cleaning between every pair of consecutive active segments.

Let $\mathcal{V}_c$ be the union of 4×1 batches and 2×2 cycles and let a_{vce} take the corresponding a^F or a^L. Cleaning order and the physical cleaning chain satisfy

$$\begin{aligned} s_v &\ge e^{\mathrm{cl}}_{c,e-1}-M(1-a_{vce}) && (e>1),\\ e_v &\le s^{\mathrm{cl}}_{ce}+M(1-a_{vce})+M(1-q^{\mathrm{cl}}_{ce}),\\ s^{\mathrm{cl,BL}}_{ce} &\ge e^{\mathrm{cl,FL}}_{ce}+r_c-M(1-q^{\mathrm{cl}}_{ce}),\\ s^{\mathrm{cl,P}}_{ce} &\ge e^{\mathrm{cl,BL}}_{ce}-M(1-q^{\mathrm{cl}}_{ce}),\\ s^{\mathrm{cl,BU}}_{ce} &\ge e^{\mathrm{cl,P}}_{ce}-M(1-q^{\mathrm{cl}}_{ce}),\\ s^{\mathrm{cl,FU}}_{ce} &\ge e^{\mathrm{cl,BU}}_{ce}+r_c-M(1-q^{\mathrm{cl}}_{ce}). \end{aligned} \tag{16}$$

Equation (16) starts a segment after the preceding cleaning and finishes it before the next cleaning. A cleaning task loads both chamber sides with dummies, rotates, processes, and unloads in reverse, thereby occupying the VTR, chamber, and four physical dummies rather than adding virtual downtime.

Dummy demand j is activated by a physical event:

$$q_j=\begin{cases} f^F_{cbl}, & j \text{ is a final-}4\times1 \text{ filling demand},\\ a^L_c, & j \text{ is a } 2\times2 \text{ head or tail demand},\\ q^{\mathrm{cl}}_{ch}, & j \text{ is a cleaning demand}. \end{cases} \tag{17}$$

Equation (17) prevents dummies from becoming an unlimited abstract filler: a physical demand exists only for a final batch, chain head/tail, or active cleaning task.

Each active demand selects one physical dummy from inventory:

$$\sum_{\tau\in\mathcal{T}_c} g_{j\tau} = q_j, \qquad j \in \mathcal{J}_c^D. \tag{18}$$

Equation (18) assigns every active demand exactly once. Mutual exclusion subsequently orders repeated uses of the same physical dummy, so it can be reused only after unloading and return to storage.

2.5. Robots, load locks, chambers, and time resources

ATR, AL, VTR, and each rotary chamber are unary resources; the upper and lower load locks each have two physical slots; each dummy in inventory is a reusable slot. An operation o requiring κ_o slots satisfies

$$\sum_{\ell\in\Gamma_o} h_{o\ell} = \kappa_o b_o. \tag{19}$$

Equation (19) assigns an active operation the required number of legal physical slots and assigns none to an inactive operation. AL, ATR, VTR, and a chamber have one slot, while load locks and dummy inventory retain physical identities.

If operations o and o' may compete for slot ℓ of resource r, then

$$\begin{aligned} s_{o'} &\ge e_o + \rho^r_{oo'} - M(1 - y^r_{oo'}) - M(1 - h_{o\ell}) - M(1 - h_{o'\ell}),\\ s_o &\ge e_{o'} + \rho^r_{o'o} - M y^r_{oo'} - M(1 - h_{o\ell}) - M(1 - h_{o'\ell}). \end{aligned} \tag{20}$$

Equation (20) makes $y^r_{oo'}$ choose one order whenever two actions use the same slot and includes empty robot travel or equipment reset through $\rho^r_{oo'}$. It applies to ATR, AL, both load locks, VTR, rotary chambers, and individual dummies, preventing concurrent robot actions and reuse before release.

An individual product or dummy stage must coincide with its selected batch, chain position, or cleaning window. For selection x and reference stage $[s^{\mathrm{ref}}, e^{\mathrm{ref}}]$,

$$\begin{aligned} -M(1 - x) \le s_o - s^{\mathrm{ref}} \le M(1 - x),\\ -M(1 - x) \le e_o - e^{\mathrm{ref}} \le M(1 - x). \end{aligned} \tag{21}$$

Equation (21) synchronizes the individual action with its physical stage when $x = 1$ and releases the candidate relation when $x = 0$, connecting pair positions, product routes, dummy demands, and chamber windows.

If pair p uses one paired transfer on robot route a, the two wafer stages are synchronized:

$$\begin{aligned} -M(1 - \gamma_{pa}) \le s_{w_1 a} - s_{w_2 a} \le M(1 - \gamma_{pa}),\\ -M(1 - \gamma_{pa}) \le e_{w_1 a} - e_{w_2 a} \le M(1 - \gamma_{pa}). \end{aligned} \tag{22}$$

Equation (22) means that $\gamma_{pa} = 1$ moves a complete pair in one non-preemptive robot action. If $\gamma_{pa} = 0$, the wafers still occupy the same robot separately and are ordered by Eq. (20).

For the entry and exit events of wafer w in restricted equipment region r,

$$0 \le s_{\mathrm{out}(w,r)} - e_{\mathrm{in}(w,r)} \le \overline{R}_{wr}. \tag{23}$$

Equation (23) limits the physical stay in an upper load lock, chamber, lower load lock, or restricted transport region. Load-lock occupation also spans placement, pump/vent, opposite-side pickup, and pressure reset, so placement by a robot is not mistaken for slot release.

2.6. Throughput-first and schedule-quality-second optimization

Let C_w be the time at which product wafer w completes the lower-load-lock return and reaches its load port. Stage one solves

$$C_w = e_{w,\mathrm{return}}, \qquad C_{\max} \ge C_w \ (w \in \mathcal{W}), \qquad \min_{\xi\in\mathcal{F}} C_{\max}. \tag{24}$$

Equation (24) measures makespan over the complete end-to-end route rather than at the last chamber operation. The feasible region $\mathcal{F}$ consists of Eqs. (1)–(23) expanded over all applicable indices.

Given stage-one incumbent $\widehat{\xi}$ and makespan $\widehat{C}$, stage two fixes the discrete structure indexed by $\mathcal{D}_{\mathrm{fix}}$ and minimizes linear schedule quality within the makespan tolerance:

$$\begin{aligned} \xi_d = \widehat{\xi}_d \quad (d \in \mathcal{D}_{\mathrm{fix}}), \qquad C_{\max} \le \widehat{C} + \varepsilon_C,\\ L_{\mathrm{sum}} = \sum_{\alpha\in\mathcal{Q}^w} \omega_\alpha, qquad L_{\max} \ge \omega_\alpha,\\ \Delta_{ceq} = (h_{ce,q+1} - h_{ceq}) - (h_{ceq} - h_{ce,q-1}),\\ v_{ceq} \ge \Delta_{ceq}, \qquad v_{ceq} \ge -\Delta_{ceq},\\ \eta_\alpha \ge \omega_\alpha - \bar{\omega}_\alpha, qquad \eta_\alpha \ge 0,\\ \min \Phi = \lambda_1 L_{\mathrm{sum}} + \lambda_2 L_{\max} + \lambda_3 \sum_{(c,e,q)\in\mathcal{I}_h} v_{ceq} + \lambda_4 \sum_{\alpha\in\mathcal{Q}^w} \eta_\alpha. \end{aligned} \tag{25}$$

Equation (25) preserves the stage-one makespan and principal discrete routes and resource orders while reducing total wait, maximum wait, interval variation within a cleaning segment, and optional soft-limit excess. The experiments use $(\lambda_1, \lambda_2, \lambda_3) = (1, 10, 25)$ and $\varepsilon_C = 10^{-6}$; the soft wait limit is disabled.

2.7. Coverage and model boundary

After index expansion, Eqs. (1)–(25) jointly describe the finite-batch dual-source problem with mixed $4 \times 1/2 \times 2$ operation, periodic cleaning, and finite dummy inventory. Equations (1)–(2) encode source interfaces: one interface is single source, while two interfaces with products and dummies assigned separately are dual source. Equations (3)–(6) encode locations, activation, durations, and precedence for product-containing PW pairs. Equations (7)–(9) encode a 4×1 full batch formed by two chamber-side PW pairs, final-batch filling, rotation, and last-in–first-out unloading. Equations (10)–(13) encode the consecutive 2×2 chain, two adjacent exposures per product-containing PW pair, dummy-PW-pair head/tail filling, and internal exchange. Equations (14)–(16) encode legal cleaning boundaries, segment capacity, and the physical cleaning chain. Equations (17)–(18) encode dummy demand and finite inventory. Equations (19)–(23) encode mutual exclusion of

robots, load locks, chambers, and dummies, paired transfer, and residency. Equations (24)–(25) encode end-to-end makespan followed by wait/cadence refinement. A pure mode is obtained by making the other mode set empty.

Every active material route has a finite precedence chain from release to return, and every pair of operations competing for one resource must be ordered. If those inequalities formed a directed cycle, summing them would require an event to occur later than itself by a positive duration, so the combination is infeasible. The resource-ordering and process-precedence constraints therefore eliminate circular waits from the schedule.

The model boundary is explicit: preprocessing fixes wafer pairing, equivalent chamber sides, and legal cleaning segments; action times are deterministic within a solve; random failures and real-time rescheduling are excluded. These choices retain the central coupling among sources, modes, dummies, cleaning, and shared resources while removing label permutations with no physical distinction.

3. Structure-aware learning-enhanced exact solution

The primal difficulty is late arrival of a competitive feasible solution; the dual difficulty is that generic cut scores do not reveal whether a cut mainly tightens discrete choices, continuous event times, or their coupling. We first construct a feasible starting point with a structure-preserving binary skeleton (SPBS), and then use RDMCT-HBS to select cuts in SCIP separation callbacks. The former improves the primal bound and the latter guides relaxation tightening, while SCIP retains responsibility for feasibility checks, valid-cut generation, bounds, and optimality proofs (Achterberg, 2009; Bestuzheva et al., 2023).

The complete workflow comprises six stages: construct and validate a start, generate candidate cuts, predict the cut count, build the cut sequence step by step, complete its structure, and resume exact search. The learning component neither generates inequalities nor changes cut coefficients. It determines the retained prefix and input order of rows already produced by SCIP.

3.1. SPBS initial solution

SPBS does not guess every event time. It first fixes the discrete skeleton that can be inferred reliably from the recipe. Product wafers are paired in input order. The 4×1 pairs are mapped sequentially to chambers, batches, and front/back positions, whereas the 2×2 pairs are assigned round-robin to chamber-chain positions. This determines batch-use, tail-batch, used-position, and last-position indicators. Cleaning counts partition each chamber sequence into process segments: 4×1 batches are assigned to their cleaning epochs, while a 2×2 chain ends its current segment with process-enabling carriers (PECs) when the cleaning threshold is reached and resumes after cleaning with a new PEC-bounded head. The skeleton therefore determines what is processed, in which chamber, and at which legal cleaning boundary; detailed PEC occupancy and all continuous event times remain governed by the MILP.

At the resource level, SPBS derives a conservative order from the same physical route. ATR first schedules 4×1 inputs and returns across chambers and stages the first 2×2 pair of a chamber immediately after that chamber's final full batch. It then advances chain inputs and returns segment by segment. Deterministic odd/even load-lock slots break slot symmetry, and the VTR order is aligned with ATR batch progression and cleaning actions. Let $\mathcal{F}_q$ be the binary variables fixed in relaxation profile q, with skeleton value $\widehat{x}_j$. The temporary model imposes

$$\ell_j^{(q)} = u_j^{(q)} = \widehat{x}_j, \qquad j \in \mathcal{F}_q. \tag{26}$$

Equation (26) fixes only the binary choices regarded as reliable in the current profile. Start, completion, and waiting times, together with resource orders outside that profile, remain free.

If the strict skeleton does not yield a feasible solution within its budget, SPBS enlarges the search space through five implemented profiles. First, `spbs_full` fixes product/batch assignment, activation, cleaning epochs, load-lock slots, ATR/AL/LL order, and VTR order. Second, `spbs_relaxed_vtr` releases the high-risk VTR order. Third, `spbs_relaxed_ll` additionally releases load-lock order. Fourth, `spbs_fixed_slots` releases ATR, AL, and all remaining resource orders but retains structural assignments and slots. Fifth, `spbs_structure_only` also releases slots, leaving only assignment, activation, and cleaning structure fixed. For instances of at least 32 wafers, the implementation begins from fixed-slot or structure-only profiles so that more time is available for feasibility search; under a tight waiting-time cap it also retains a fully unfixed `spbs_unrestricted` attempt.

Every profile rebuilds a temporary model, sets the objective to zero, uses feasibility emphasis, aggressive heuristics, fast presolve, no separation, and stops at the first solution. Once a physical schedule is found, the program reconstructs the full model, fixes its principal discrete and continuous values, releases only secondary stability auxiliaries, completes them, and invokes the complete SCIP solution check. Only a checked high-precision solution file is passed to the production model as an incumbent. No SPBS bound is transferred, so the production solve remains free to improve the schedule and complete the global proof. In the formal benchmark SPBS is precomputed offline; its metadata records the accepted profile and all attempts, and a SHA-256 hash of the serialized LP prevents a start from being reused on a different model. Its construction cost is excluded from reported PDI and main-solve time.

3.2. HEM baseline and role-aware representation

Prior learning-based cut selectors address generic candidate sets with reinforcement learning, learned ranking, and look-ahead imitation (Tang et al., 2020; Huang et al., 2022; Paulus et al., 2022). HEM further identifies three coupled decisions—which cuts, how many cuts, and in what order—and handles them with a high-level count policy and a low-level sequence policy (Wang et al., 2023b). We retain HEM's hierarchical decomposition and pointer decoding and use its 13D implementation as the direct baseline. Training instances, budgets, random seeds, candidate limits, and hidden dimensions are shared; only the input representation and post-selection completion are changed.

The 13D HEM candidate vector comprises objective parallelism, efficacy, normalized support, integral-support ratio, normalized violation, the mean/maximum/minimum/standard deviation of row coefficients, and the corresponding four objective-coefficient statistics. These features describe immediate LP quality and numerical scale. They do not distinguish two similarly efficacious rows when one acts on discrete route choices and the other on continuous event times. In the present model, that distinction corresponds directly to changing a process/resource combination versus tightening the timing of an existing combination.

Starting from the 13 generic HEM features, variables are partitioned by solver metadata into binary, general integer, objective-active continuous, continuous auxiliary, and other roles. The coefficient mass of role g in cut i is

$$p_{ig} = \frac{\sum_{j:x_j \in g} |a_{ij}|}{\sum_j |a_{ij}| + \varepsilon}. \tag{27}$$

Equation (27) is the fraction of absolute cut-coefficient mass carried by a mathematical variable role. Beyond the five role masses, the added features are mass on variables with two finite global bounds, positive-coefficient mass, discrete–continuous coupling $4(p_{iB} + p_{iI})(p_{iC_{\text{obj}}} + p_{iC_{\text{aux}}})$, the largest non-residual role mass, and normalized role entropy. The resulting vector has 23 dimensions. Roles are identified solely from SCIP variable types, objective coefficients, and global bounds; neither variable names nor equipment labels are read, so variable renaming leaves the representation unchanged.

At the network input, layer normalization is followed by separate projections of the 13 generic and 10 role-aware components. A sigmoid gate generated from the full vector fuses the two projections. This adapter can scale immediate cut quality and variable-role composition separately while retaining HEM's single-layer LSTM encoder, 128-dimensional hidden state, one glimpse, and low-level pointer decoder (Vinyals et al., 2015). The changes from HEM are therefore explicit: the role representation, the grouped gated input adapter, and the policy-anchored role-submodular completion described below.

3.3. Stepwise selection from candidate pool to ordered cuts

At separation callback t, the method executes the following sequence.

Step 1: form the candidate pool. SCIP first generates and validates candidate rows C_t with its own separators. To avoid traversing the complete sparse support of thousands of rows, the method uses SCIP's inexpensive efficacy accessor to retain at most 128 candidates. Ties are broken by original index for deterministic input. Forced cuts remain under solver control and are not placed in the learned ranking.

Step 2: compute the state. The 13 HEM and 10 role-aware features are extracted for every row in the pool, while role masses, dominant role, and structural entropy are retained as metadata. This candidate set is the state S_t.

Step 3: determine the retained count. The high-level policy outputs a tanh-bounded action in $[-1, 1]$, which is mapped to a retained fraction ρ_t. The fraction is then clipped by the global 0.2 cap, SCIP's round-specific limit, and the implementation cap of 16 cuts, producing k_t. When enough candidates exist, at least two rows are retained to prevent repeated callbacks from degenerating to an empty selection.

Step 4: construct the candidate order. At low-level step r, the pointer policy conditions on S_t and the selected prefix $i_{1:r-1}$ to score all remaining candidates. Selected indices are masked with negative infinity and cannot be repeated. The joint high- and low-level policy is

$$\pi_\theta(k_t, i_{1:k_t} \mid S_t) = \pi_\theta^H(k_t \mid S_t) \prod_{r=1}^{k_t} \pi_\theta^L(i_r \mid S_t, i_{1:r-1}). \tag{28}$$

Equation (28) means that the high level determines the sequence length and the low level then chooses distinct candidates within that length. Each earlier cut changes both the remaining action set and the decoder state used by later probabilities.

Step 5: beam decoding. The low-level policy samples stochastically during training. At test time, beam search of width three is used. At step r, every partial sequence is extended by every unselected candidate; joint log-probabilities are accumulated, and the three highest-scoring items over all parent-beam–next-candidate pairs are retained. After k_t steps, the best sequence is returned. Unlike one-step greedy decoding, the beam retains an initially lower-ranked prefix whose later joint probability can be higher. This fixed-width neural construction follows the broader use of beam decoding in neural combinatorial optimization (Choo et al., 2022).

Step 6: role-submodular completion. A sequence based only on joint probability can still repeatedly cover one variable role. The first $\lceil 0.25k_t \rceil$ pointer candidates are retained as policy anchors. The remaining policy order is merged with a heuristic tail sorted by instantaneous structural quality q_i, and at most $2k_t$ rows form expansion pool $\mathcal{R}_t$. For $S \subseteq \mathcal{R}_t$, define

$$\begin{aligned} F_t(S) = {} & \sum_{i \in S} (0.35 q_i + 0.20 r_i) + 0.25 \sum_{g \in \mathcal{G}} d_g \sqrt{\sum_{i \in S} p_{ig}} \\ & + \frac{0.20}{|\mathcal{R}_t|} \sum_{u \in \mathcal{R}_t} \max_{i \in S} s_{ui}, \end{aligned} \tag{29}$$

where r_i is the normalized pointer-rank prior, d_g is the demand share of role g in the expansion pool, and s_{ui} is clipped cosine similarity on generic-quality and role-mass descriptors. The score q_i combines efficacy, violation, integral support, recognized-role ratio, dominant-role ratio, and structural entropy. In Eq. (29), the first term retains instantaneous quality and the learned order; the square-root coverage term gives diminishing marginal gain to repeated role mass; and the final term makes the selected cuts represent the main structures in the pool. Starting from the anchors, the method recomputes the marginal gain of every unselected candidate and adds the largest until k_t cuts have been selected. The nonnegative modular, concave-over-modular, and facility-location terms form a monotone submodular objective, so the residual greedy completion after fixing anchors and the expansion pool inherits the standard $1 - 1/e$ approximation property (Nemhauser et al., 1978).

Finally, the k_t selected rows are placed at the front of the returned array in the chosen order, while all other candidates remain as an unselected suffix. SCIP adds only the prefix to the current LP. One callback therefore completes six operations: inexpensive truncation, feature extraction, count prediction, sequence decoding, structural completion, and return to SCIP.

Algorithm 1: Cut selection by parallel hierarchical REINFORCE and beam search

Algorithm steps

Input: SCIP-validated rows C_t, high-level policy π_θ^H, low-level pointer policy π_θ^L, beam width $B = 3$, and the current SCIP cut limit.
Output: Ordered cut prefix $S_t^\star = (i_1, \ldots, i_{k_t})$ and the unselected suffix returned to SCIP.
1. Train the policies. Two workers collect rollouts in parallel with terminal reward $R = -\text{PDI}$; a central learner applies hierarchical REINFORCE with separate EMA baselines and synchronous Adam updates. Fix the parameters for formal inference.
2. Form the pool. Sort by efficacy, break ties by original index, and retain at most 128 rows.
3. Extract the state. Compute 13 HEM features, ten role-aware features, and role-mass metadata for every row.
4. Determine the count. Predict the retained fraction with π_θ^H and obtain k_t after the 0.2 fraction cap, SCIP round limit, and 16-cut implementation cap.
5. Initialize the beam. Set $\mathcal{B}_0$ to the empty sequence with log probability zero.
6. Expand. For $r = 1, \ldots, k_t$, extend every prefix in $\mathcal{B}_{r-1}$ by every unselected candidate, mask repeated indices, and accumulate pointer log probability.
7. Prune. Retain the three highest joint-probability parent–candidate extensions as $\mathcal{B}_r$; after the loop, take the leading sequence in $\mathcal{B}_{k_t}$.
8. Fix anchors. Retain its first $\lceil 0.25 k_t \rceil$ rows as policy anchors A_t.
9. Form the expansion pool. Merge the remaining pointer order with structural-quality order, remove duplicates, and keep at most $2k_t$ rows as $\mathcal{R}_t$.
10. Complete submodularly. Set $S \leftarrow A_t$; while $|S| < k_t$, add the $i \in \mathcal{R}_t \setminus S$ maximizing $F_t(S \cup \{i\}) - F_t(S)$.
11. Return. Return $S_t^\star$ in anchor-then-completion order; leave all other rows in the suffix and outside the current LP.

3.4. Reinforcement-learning training and exact-solver interface

One time-limited SCIP solve forms a rollout, with its terminal PDI used as the cost. The current policy is executed in every eligible cut-selection callback. For replay, the agent records the high-level fraction and low-level sampled sequence at the first valid candidate set of that rollout, and the PDI of the complete solve is returned to both policies. Writing $R = -\text{PDI}$ as a reward, the low-level estimator is

$$\widehat{\nabla_\theta J} = \frac{1}{B} \sum_{b=1}^{B} (R_b - \bar{R}_{\text{EMA}}) \nabla_\theta \log \pi_\theta(\tau_b), \qquad \bar{R}_{\text{EMA}} \leftarrow 0.9 \bar{R}_{\text{EMA}} + 0.1 \bar{R}. \tag{30}$$

Equation (30) is batch REINFORCE with an exponential-moving-average baseline; B is the rollout batch and τ_b is the recorded cut sequence in rollout b. Equivalently, the code minimizes PDI-cost advantage times log probability. Low-level log probability is averaged over sequence length so that a longer cut sequence does not produce a larger gradient merely because it contains more terms. The high-level policy uses the same PDI cost and an independent EMA baseline and is updated every two epochs; both gradient norms are clipped at 2. The implementation adopts the A3C idea of parallel environments to reduce sample correlation, but a central learner updates parameters synchronously after the batch is collected. It is therefore parallel hierarchical REINFORCE rather than asynchronous actor–critic (Williams, 1992; Mnih et al., 2016).

Training uses five instances and five random seeds. Each seed has two workers, each completing four 30-s/1024-MB rollouts per epoch, for eight rollouts per epoch and 60 epochs. Initial Adam learning rates are 10^{-4} for the low-level policy and 5×10^{-4} for the high-level policy and are multiplied by 0.96 every ten epochs. Training uses stochastic decoding and formal inference uses beam width three. Every method–seed combination uses the fixed epoch-60 parameters without benchmark- or test-set checkpoint selection. HEM-13D, feature-only 23D, and the complete 23D method are trained separately so that weights are never mixed across input semantics or completion environments.

RDMCT-HBS only orders and selects valid rows already generated by SCIP, and SPBS submits only solutions checked in the complete model. Neither changes the objective, constraints, integrality, or bound logic. The learned components therefore change the search path and finite-time bound convergence, whereas feasibility and optimality retain the same meaning as in default SCIP.

4. Computational results and analysis

4.1. Experimental design and metrics

Seven complementary evidence sets are used. Four small exact solves check internal implementation consistency. A 2700-run nominal study compares seven solver-guidance configurations on 20 manufacturing instances; its 150-run exact subset supplies the equal-wafer-count mode comparison. A 960-run matched study evaluates precomputed SPBS input, and 1500 runs compare three equal-total-budget workflows. A 300-run factorial design associates scale, recipe composition, and process duration with difficulty; 80 one-factor runs assess operation under parameter perturbations; and 729 distribution-shift runs test feasible-schedule delivery on 40–64-wafer batches. The six formal benchmark groups contain 6269 runs and the four exact checks bring the total to 6273. All 6041 saved incumbents pass SCIP and an independent post-solve check against the same serialized MILP; no invalid schedule is found.

The MILP, SCIP version, instances, and random seeds are frozen, and each comparison family uses common time and memory limits. The main metric is the normalized primal–dual gap integral over a common horizon $[0, T]$, $\text{PDI} = \int_0^T g(t)\,\mathrm{d}t$; lower values mean that the bound gap closes earlier. PDI can be zero only if the gap is zero for almost the entire timed horizon. A zero terminal gap instead means that optimality is proved at termination and does not imply zero PDI. The manufacturing instance is the inferential unit: random repeats are first aggregated within an instance, after which paired instance differences use nonparametric 95% bootstrap intervals, Wilcoxon signed-rank tests, and Holm correction within each registered analysis family (Gleixner et al., 2021). Time-limited runs remain in the summaries so that random seeds are not treated as independent manufacturing evidence. Full hardware, budgets, tolerances, instance lists, and hyperparameters are provided in the supplementary reproducibility package.

4.2. Processing mode and two-stage operational value

Steady-state studies of multi-space CTs commonly evaluate equipment efficiency through cycle time or throughput (Gu et al., 2024). For the finite batches studied here, the experiment reports both exact makespan and the equivalent throughput derived from it.

Figure 4 compares pure 4×1, pure 2×2, and 1:1 mixed operation at 8 and 16 wafers under the same topology, transport/cleaning parameters, processing scale, and 600-s budget. All six configurations reach zero terminal gap; each result is confirmed by the complete cross of five solver and five training seeds.

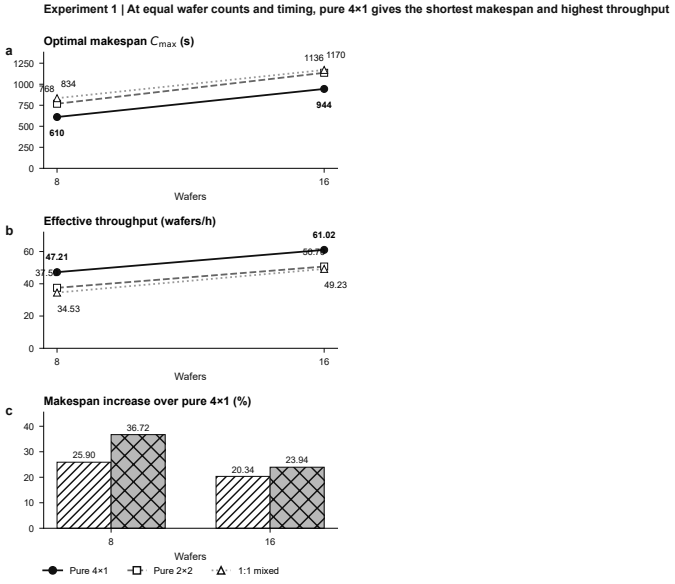


Figure 4: Mode efficiency at equal wafer counts. a, exact optimal $C_{\max}$; b, equivalent throughput $3600n/C_{\max}$; c, makespan increase of pure 2×2 and mixed operation over pure 4×1. Each point represents 25 exact repetitions, 150 runs in total; values are labeled directly.

At 8 wafers, the three makespans are 610, 768, and 834 s and the equivalent throughputs are 47.21, 37.50, and 34.53 wafers/h. At 16 wafers, the corresponding values are 944, 1136, and 1170 s and 61.02, 50.70, and 49.23 wafers/h. Thus, when the recipe does not prescribe a mode and exposure times are equal, pure 4×1 achieves both the shortest makespan and the highest equivalent throughput for the 8- and 16-wafer batches.

Waiting-time constraints can propagate to manufacturing-system output and control performance (Lee et al., 2018). For the same 600-s total budget, the complete two-stage workflow assigns 540 s to the main solve and up to 60 s to refinement, whereas the one-stage control assigns all 600 s to the main solve. Figure 5 shows that the two-stage workflow reduces total route wait, maximum wait, cadence coefficient of variation, and cadence deviation by 51.91%, 68.89%, 39.70%, and 94.05%, respectively. All four paired instance differences remain significant after Holm correction, demonstrating that the two-stage workflow markedly improves waiting and cadence quality under the same total budget.

The quadratic-objective control and the complete linear refinement both use the 540+up-to-60-s split. Linear refinement yields a cadence deviation of 22.04, 63.38% below the quadratic profile's 60.18, and the matched difference remains significant after Holm correction ($p = 0.0377$); total route wait, maximum wait, and cadence coefficient of variation are similar overall. The linear objective therefore produces a more regular processing cadence under the same budget.

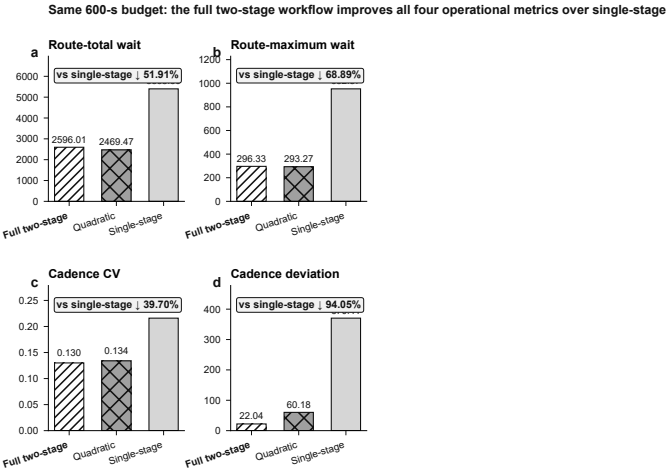


Figure 5: Final-schedule quality under the same 600-s total budget. a–d, total route wait, maximum route wait, cadence coefficient of variation, and cadence deviation; 1500 runs in total, with observed means labeled above the bars.

4.3. SPBS and structure-aware cut selection

The paired SPBS study uses 48 instances and ten paired seeds per instance. Relative to the no-start condition, precomputed SPBS increases incumbent availability from 51.67% to 100%, a gain of 48.33 percentage points (95% interval [34.58, 62.29]). Mean PDI falls from 27473.05 to 20278.78, a decrease of 7194.27 or 26.19% (95% interval [5145.78, 9317.61]). Both paired differences remain significant after Holm correction ($p = 2.55 \times 10^{-5}$ and $p = 3.46 \times 10^{-8}$). These results show that SPBS substantially increases finite-time access to complete feasible schedules and improves primal–dual search efficiency.

In the 2700 nominal runs, HEM-13D is the direct baseline and RDMCT-HBS uses the same training instances, random seeds, candidate cap, hidden dimension, SPBS incumbent, and solve budget. Relative to HEM-13D, RDMCT-HBS reduces mean PDI from 16823.18 to 16585.70 (1.41%), mean solution time from 219.65 to 218.32 s (0.61%), and mean terminal gap from 127.24% to 126.01% (0.97%), while raising the proven-optimal rate from 62.4% to 63.2% (0.80 percentage points). All 500 of its runs return feasible solutions (Fig. 7). The other five configurations provide default-solver, alternative-policy, and component-ablation context. Replacing the 13 generic features with the 23 role-aware representation gives the largest mean improvement, followed by role completion and beam decoding in the complete method.

Against the direct HEM baseline, RDMCT-HBS improves PDI, solution time, terminal gap, and proven-optimal rate. It also has the lowest mean PDI and mean solution time among

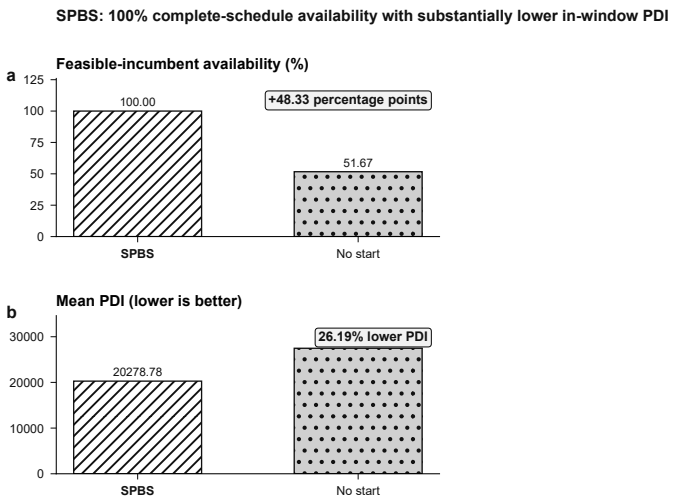


Figure 6: Effect of precomputed SPBS on primal search. a, incumbent availability; b, mean PDI. Each condition has $n = 480$; observed values are labeled directly.

all seven configurations. The result shows that role-aware representation, policy-anchored completion, and beam decoding improve the finite-time search trajectory while retaining HEM's hierarchical decision structure.

4.4. Scale, recipe composition, and computational difficulty

The factorial design contains $4 \times 5 \times 3 = 60$ wafer-count–recipe-composition–process-scale cells with five solver seeds per cell, for 300 runs. Figure 8 shows a sharp transition. All 75 runs at both 8 and 16 wafers prove optimality, with mean PDI values of 28.13 and 1876.54 and mean solution times of 0.34 and 33.86 s. At 24 and 32 wafers, the proven-optimal rates fall to 36% and 20%, mean PDI rises to 31155.94 and 32959.89, and mean solution time reaches 408.55 and 433.20 s.

The prespecified PDI factorial model has $R^2 = 0.998$. Under the 1:1 mixed recipe, the additional interaction coefficients at 24 and 32 wafers are 37729.48 and 40788.63, while the 95% intervals for the process-scale main effect and all prespecified process-scale interactions include zero. Over the tested 0.9–1.1 process-scale range, computational difficulty is driven primarily by the interaction between batch size and mixed-flow composition, with a comparatively small process-scale effect.

4.5. Operation under parameter perturbations

Prior CT studies show that cleaning actions and their triggering conditions alter robot-task sequences, cycle time, and wafer delay (Yu and Lee, 2020; Li et al., 2022). The one-factor sensitivity study therefore covers eight prescribed perturbations of transport, processing, cleaning interval, and vacuum-domain dummy inventory, for 80 runs. Every run returns a solver-checked feasible schedule and 20 runs prove optimality; scenario mean PDI ranges from 28899.41 to 42478.32. A longer cleaning interval gives the lowest mean PDI and slower transport the highest, showing that cleaning frequency and shared-robot transport cadence materially shape search difficulty. The

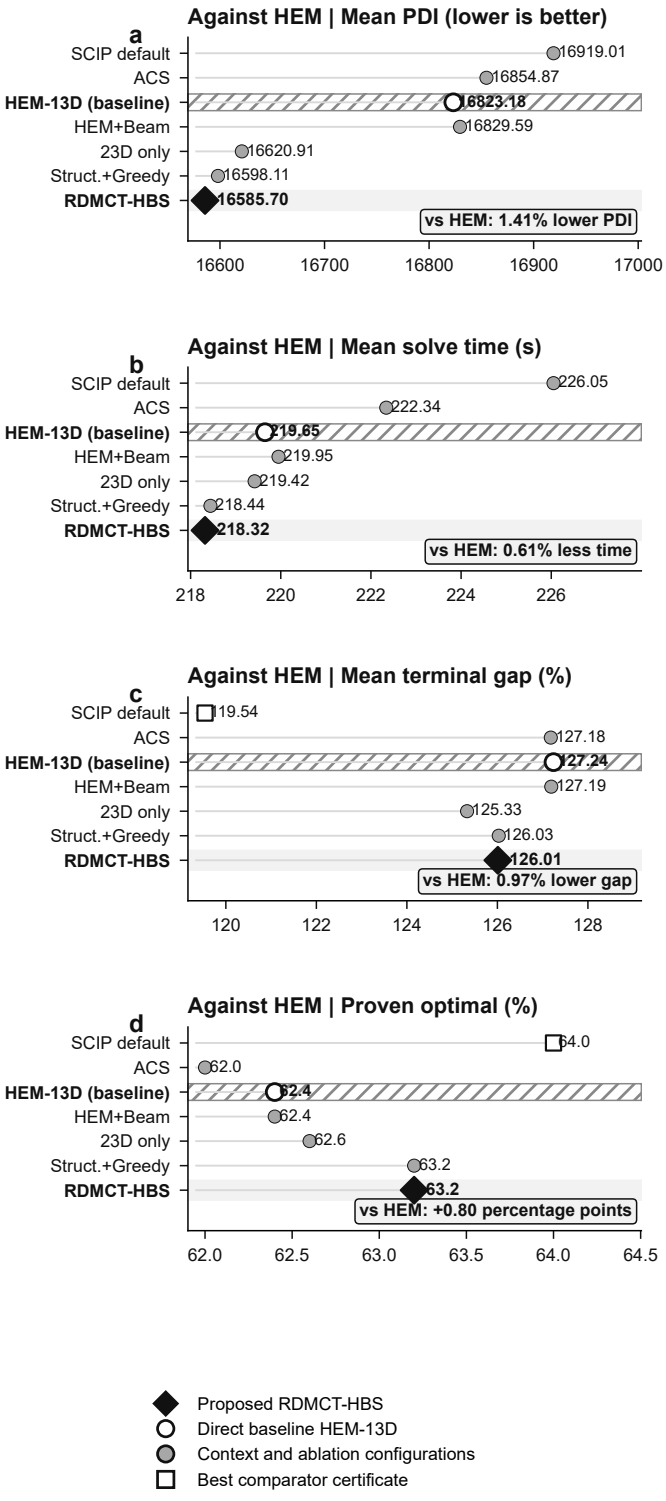


Figure 7: Nominal comparison of seven solver-guidance configurations sharing SPBS. HEM-13D is the direct baseline and RDMCT-HBS is the primary comparator; the other five configurations provide default-solver, alternative-policy, and component-ablation context. Panels report mean PDI, mean solution time, mean terminal gap, and proven-optimal rate over 2700 runs, with values and improvements over HEM labeled directly. The black diamond marks RDMCT-HBS and the hatched open circle marks HEM.

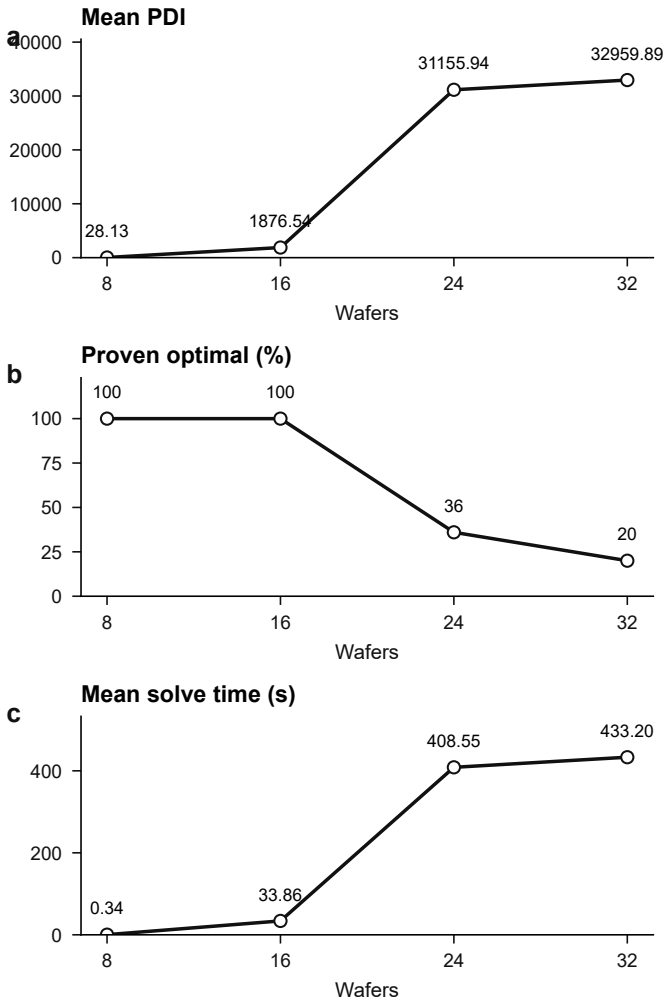


Figure 8: Scale transition in the 300-run factorial experiment. a, mean PDI; b, proven-optimal rate; c, mean solution time. Each wafer count contains 75 runs and observed values are labeled directly.

framework nevertheless returns a feasible schedule in every prescribed perturbation scenario.

4.6. Feasible-schedule delivery under distribution shift

The distribution-shift suite enlarges batches to 40–64 wafers and changes mixed-flow composition, producing 729 runs over nine configurations. All 729 runs return an incumbent that passes SCIP and independent post-solve checks, showing that the SPBS–MILP–SCIP solution pipeline reliably delivers verifiable schedules at the larger shifted scale.

In the distribution-shift experiment, adaptive cut selection records the lowest mean PDI (70915.54) and terminal gap (160.78%). The corresponding values are 73919.15 and 170.02% for RDMCT-HBS, and 73670.99 and 171.35% for default SCIP. Method rankings change on the larger instances, while the shared SPBS–MILP–SCIP pipeline consistently returns a checked feasible schedule in all 729 runs.

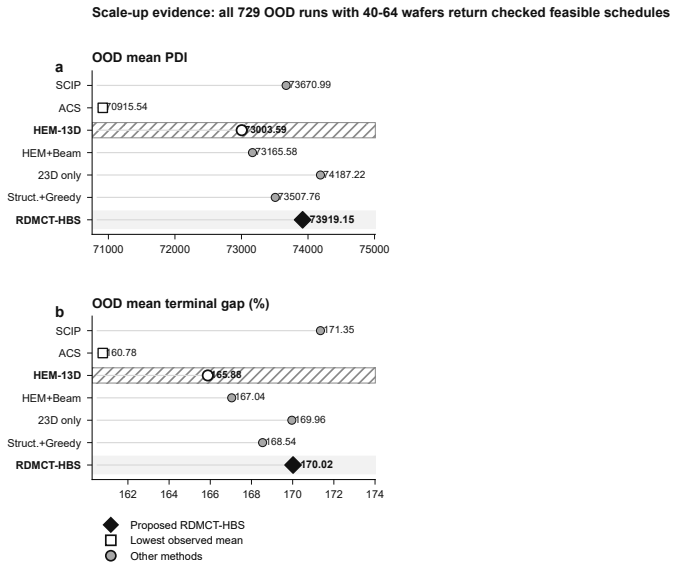


Figure 9: Distribution-shift experiment with 40–64 wafers. a, instance-first mean PDI for seven configurations; b, mean terminal gap. All 729 runs return checked feasible schedules; observed values are labeled directly.

4.7. Industrial interpretation and integrated findings

The experiments support three industrial conclusions. When the recipe does not prescribe a mode, pure 4×1 provides the highest throughput for the 8- and 16-wafer batches. Under the same total budget, the two-stage workflow significantly improves waiting and cadence quality. SPBS and structure-aware cut selection improve the primal start and finite-time search trajectory, respectively. The factorial, sensitivity, and distribution-shift studies further identify batch size and mixed-flow composition as the main drivers of computational difficulty and show that the framework continues to deliver checked schedules under parameter perturbations and on 40–64-wafer instances.

5. Conclusions

This study develops a finite-horizon two-stage MILP for dual-source rotary-chamber semiconductor cluster tools that unifies 4×1 parallel processing, 2×2 chained processing, reusable dummy wafers, cleaning, and shared transport. A two-wafer PW pair is the physical transport and chamber-occupancy unit and can be P+P, D+D, or P+D. The model makes a 4×1 full batch process synchronously once and makes every product wafer in a 2×2 chain undergo two processes in adjacent cycles. The formulation is coupled with an SPBS initial solution and the structure-aware hierarchical cut selector RDMCT-HBS. Exact mode comparisons show that pure $4{\times}1$ achieves the shortest makespan and highest equivalent throughput for both 8- and 16-wafer batches. Under the same 600-s total budget, the two-stage workflow reduces total route wait, maximum route wait, cadence coefficient of variation, and cadence deviation by 51.91%, 68.89%, 39.70%, and 94.05%, respectively. SPBS raises finite-time feasible-solution availability from 51.67% to 100% and reduces mean PDI by 26.19%. Relative to the direct HEM-13D baseline, RDMCT-HBS reduces mean PDI, solution

time, and terminal gap by 1.41%, 0.61%, and 0.97%, and raises the proven-optimal rate by 0.80 percentage points. The factorial experiment identifies the interaction between batch size and mixed-recipe composition as the main driver of computational difficulty, and all 729 distribution-shift runs on 40–64-wafer instances return solver-checked feasible schedules.

These results show that embedding equipment structure across MILP formulation, two-stage schedule-quality optimization, primal initialization, and cut selection improves both manufacturing schedule quality and finite-time exact-search efficiency. Future work will extend the source-interface model to stochastic arrivals, equipment disturbances, and online rescheduling, and will assess SPBS construction, rolling optimization, and line-level deployment jointly.

CRediT authorship contribution statement

Lincong Wei was responsible for Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, and Writing – review and editing. Lizhen Shao was responsible for Conceptualization, Methodology, Formal analysis, Validation, Writing – review and editing, and Project administration. Wei Wang was responsible for Formal analysis, Investigation, Writing – review and editing, and Supervision. All authors reviewed and approved the final manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

The data, mathematical instances, trained models, and source code supporting the findings are available from the corresponding author on reasonable request. A permanent public archive will be cited in the version of record.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI ChatGPT and Codex to improve language and readability, check LaTeX consistency, and assist with software debugging. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

Pan, C.R., Zhou, M.C., Qiao, Y., Wu, N.Q., 2018. Scheduling cluster tools in semiconductor manufacturing: Recent advances and challenges. IEEE Transactions on Automation Science and Engineering 15, 2, 586–601. doi:10.1109/TASE.2016.2642997.

Gu, L., Wu, N., Qiao, Y., Zhang, S., Li, T., 2024. Scheduling cluster tools with multi-space process modules and a multi-finger-arm robot in wafer fabrication subject to wafer residency time constraints. Applied Sciences 14, 9490. doi:10.3390/app14209490.

Achterberg, T., 2009. SCIP: Solving constraint integer programs. Mathematical Programming Computation 1, 1–41. doi:10.1007/s12532-008-0001-1.

Bestuzheva, K., Besançon, M., Chen, W.K., Chmiela, A., Donkiewicz, T., van Doornmalen, J., Eifler, L., Gaul, O., Gamrath, G., Gleixner, A., Gottwald, L., Graczyk, C., Halbig, K., Hoen, A., Hojny, C., van der Hulst, R., Koch, T., Lübbecke, M.E., Maher, S.J., Matter, F., Mühmer, E., Müller, B., Pfetsch, M.E., Schlösser, F., Serrano, F., Shinano, Y., Sofranac, B., Turner, M., Vigerske, S., Wegscheider, F., Wellner, P., Weninger, D., Witzig, J., 2023. Enabling research through the SCIP optimization suite 8.0. ACM Transactions on Mathematical Software 49, 22:1–22:21. doi:10.1145/3585516.

Choo, J., Kwon, Y.D., Kim, J., Jae, J., Hottung, A., Tierney, K., Gwon, Y., 2022. Simulation-guided beam search for neural combinatorial optimization, in: Advances in Neural Information Processing Systems. URL: https://papers.nips.cc/paper_files/paper/2022/hash/39b9b60f0d149eabd1fff2d7c7d5afc4-Abstract-Conference.html, doi:10.52202/068431-0637.

Gleixner, A., Hendel, G., Gamrath, G., Achterberg, T., Bastubbe, M., Berthold, T., Christophel, P., Jarck, K., Koch, T., Linderoth, J., Lubbecke, M.E., Mittelmann, H.D., Ozyurt, D., Ralphs, T.K., Salvagnin, D., Shinano, Y., 2021. MIPLIB 2017: Data-driven compilation of the 6th mixed-integer programming library. Mathematical Programming Computation 13, 443–490. doi:10.1007/s12532-020-00194-3.

Gu, L., Wu, N., Qiao, Y., Zhang, S., 2024. Scheduling of Multi-finger-robotic Cluster Tools with Multi-space Process Modules, in: 2024 International Conference on Networking, Sensing and Control (ICNSC), IEEE. doi:10.1109/ICNSC62968.2024.10760109.

Hong, T.Y., Hsu, C.Y., Chen, C.L., 2026. Simulation-based deep reinforcement learning for process time optimization in semiconductor cluster tool and an empirical study. Computers & Industrial Engineering 211, 111698. doi:10.1016/j.cie.2025.111698.

Huang, Z., Wang, K., Liu, F., Zhen, H.L., Zhang, W., Yuan, M., Hao, J., Yu, Y., Wang, J., 2022. Learning to select cuts for efficient mixed-integer programming. Pattern Recognition 123, 108353. doi:10.1016/j.patcog.2021.108353.

Kim, H., Kim, H.J., Lee, J.H., Lee, T.E., 2013. Scheduling dual-armed cluster tools with cleaning processes. International Journal of Production Research 51, 3671–3687. doi:10.1080/00207543.2012.758392.

Kim, H.J., Lee, J.H., 2025. Scheduling cluster tools for concurrent processing: Deep reinforcement learning with adaptive search. IEEE Transactions on Automation Science and Engineering 22, 3783–3796. doi:10.1109/TASE.2024.3399818.

Lee, J.H., Zhao, C., Li, J., Papadopoulos, C.T., 2018. Analysis, design, and control of Bernoulli production lines with waiting time constraints. Journal of Manufacturing Systems 46, 208–220. doi:10.1016/j.jmsy.2018.01.001.

Li, C., Yang, F., Zhen, L., 2022. Efficient scheduling approaches to time-constrained single-armed cluster tools with condition-based chamber cleaning operations. International Journal of Production Research 60, 3555–3568. doi:10.1080/00207543.2021.1926568.

Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T.P., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning, in: Proceedings of the 33rd International Conference on Machine Learning, PMLR. pp. 1928–1937. URL:

https://proceedings.mlr.press/v48/mniha16.html.

Nemhauser, G.L., Wolsey, L.A., Fisher, M.L., 1978. An analysis of approximations for maximizing submodular set functions—i. Mathematical Programming 14, 265–294. doi:10.1007/BF01588971.

Paulus, M.B., Zarpellon, G., Krause, A., Charlin, L., Maddison, C., 2022. Learning to cut by looking ahead: Cutting plane selection via imitation learning, in: Proceedings of the 39th International Conference on Machine Learning, PMLR. pp. 17584–17600. URL: https://proceedings.mlr.press/v162/paulus22a.html.

Perkinson, T.L., McLarty, P.K., Gyurcsik, R.S., Cavin, R.K., 1994. Single-wafer cluster tool performance: An analysis of throughput. IEEE Transactions on Semiconductor Manufacturing 7, 369–373. doi:10.1109/66.311340.

Qiao, Y., Wu, N.Q., Zhou, M.C., 2012. Petri net modeling and wafer sojourn time analysis of single-arm cluster tools with residency time constraints and activity time variation. IEEE Transactions on Semiconductor Manufacturing 25, 432–446. doi:10.1109/TSM.2012.2199338.

Qiao, Y., Wu, N., Zhou, M.C., 2014. Scheduling of dual-arm cluster tools with wafer revisiting and residency time constraints. IEEE Transactions on Industrial Informatics 10, 286–300. doi:10.1109/TII.2013.2272702.

Qiao, Y., Lu, Y., Li, J., Zhang, S., Wu, N., Liu, B., 2022. An efficient binary integer programming model for residency time-constrained cluster tools with chamber cleaning requirements. IEEE Transactions on Automation Science and Engineering 19, 1757–1771. doi:10.1109/TASE.2021.3122576.

Rostami, S., Hamidzadeh, B., 2002. Optimal scheduling techniques for cluster tools with process-module and transport-module residency constraints. IEEE Transactions on Semiconductor Manufacturing 15, 341–349. doi:10.1109/TSM.2002.801379.

Tang, Y., Agrawal, S., Faenza, Y., 2020. Reinforcement learning for integer programming: Learning to cut, in: Proceedings of the 37th International Conference on Machine Learning, PMLR. pp. 9367–9376. URL: https://proceedings.mlr.press/v119/tang20a.html.

Vinyals, O., Fortunato, M., Jaitly, N., 2015. Pointer networks, in: Advances in Neural Information Processing Systems, pp. 2692–2700.

Wang, J., Hu, H., Pan, C., Zhou, Y., Li, L., 2020. Scheduling dual-arm cluster tools with multiple wafer types and residency time constraints. IEEE/CAA Journal of Automatica Sinica 7, 776–789. doi:10.1109/JAS.2020.1003150.

Wang, J., Liu, C., Zhou, M.C., Abusorrah, A., 2025. Virtual cell-based scheduling approach to single-robotic-arm cluster tools subject to wafer residency time constraints. IEEE Transactions on Automation Science and Engineering 22, 240–251. doi:10.1109/TASE.2024.3349380.

Wang, Z., Li, X., Wang, J., Kuang, Y., Yuan, M., Zeng, J., Zhang, Y., Wu, F., 2023b. Learning cut selection for mixed-integer linear programming via hierarchical sequence model, in: Proceedings of the Eleventh International Conference on Learning Representations. URL: https://openreview.net/forum?id=Zob4P9bRNcK.

Williams, R.J., 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning 8, 229–256. doi:10.1007/BF00992696.

Wu, N.Q., Zhou, M.C., 2010. A closed-form solution for schedulability and optimal scheduling of dual-arm cluster tools with wafer residency time constraint based on steady schedule analysis. IEEE Transactions on Automation Science and Engineering 7, 303–315. doi:10.1109/TASE.2008.2008633.

Yang, F., Li, C., Zhen, L., Zhang, C., Wu, N., 2025. Scheduling residency time-constrained single-armed cluster tools with two-wafer types via mixed integer programming model. Expert Systems with Applications 288, 128209. doi:10.1016/j.eswa.2025.128209.

Yu, T.S., Lee, T.E., 2020. Wafer delay analysis and control of dual-armed cluster tools with chamber cleaning operations. International Journal of Production Research 58, 434–447. doi:10.1080/00207543.2019.1593547.